

Design Principles and Metrics

Software Evolution and Re-
Engineering CS4118
Lect 4, 5

Design principles in Software Development

- **Design principles** are **guidelines and best practices** that help developers create software that is
 - Easy to understand
 - Easy to maintain
 - Easy to extend
 - Less error-prone
- They are **not rules**, but proven approaches that improve code quality over time.

Design Principles in Re-Engineering

- Adopt a divide and conquer approach
- Design with abstraction
- Design for good encapsulation
- Design for **higher cohesion** and **low coupling**
- Design for **re-usability**
- Design for flexibility
- Reuse existing designs where possible

Example of Design Principles

- Single Responsibility Principle (SRP)
 - A class should have only one reason to change.
- Open-closed Principle
 - Code should be open for extension and closed for modification

Before and After SRP applied

```
class Report {  
    void generateReport() {}  
    void saveToFile() {}  
    void sendEmail() {}  
}
```

```
class ReportGenerator {}  
class ReportSaver {}  
class ReportMailer {}
```

Before and after OCP Applied

```
class DiscountCalculator {  
    double calculateDiscount(String customerType, double amount) {  
        if (customerType.equals("Regular")) {  
            return amount * 0.05;  
        } else if (customerType.equals("Premium")) {  
            return amount * 0.10;  
        }  
        return 0;  
    }  
}
```

```
interface Discount {  
    double apply(double amount);  
}  
  
class RegularDiscount implements Discount {  
    public double apply(double amount) {  
        return amount * 0.05;  
    }  
}  
  
class PremiumDiscount implements Discount {  
    public double apply(double amount) {  
        return amount * 0.10;  
    }  
}  
  
class DiscountCalculator {  
    double calculate(Discount discount, double amount) {  
        return discount.apply(amount);  
    }  
}
```

Code and Design Metrics

- Object-oriented (Class-based)

 - CK Metrics (Chidamber & Kemerer)

 - LK Metrics (Lorenz & Kidd)

 - MOOD Metrics (Harrison, Counsell & Nithi)

- Others

 - McCabe

 - Halstead

 - Other tool-oriented metrics

 - Relies on static code analysis

CK Metrics

- Depth of inheritance Tree (DIT)
- Number of Children (NOC)

Reusability

- Weighted methods per class (WMC)
- Lack of cohesion in methods (LCOM)

Cohesion

- Coupling between objects (CBO)
- Response for a Class (RFC)

Coupling

Depth of inheritance Tree (DIT)

- DIT is the length of the longest path from a class to the root class in the inheritance hierarchy.
- What it Measures
 - Inheritance depth
 - Design complexity
 - Method reuse
- High DIT → more reuse but harder to understand and maintain
- Low DIT → simpler design but less reuse

NOC (Number of Children)

- NOC is the number of immediate subclasses of a class.
- What it Measures
 - Reuse of a base class
 - Impact of changes in parent class
- High NOC → base class is highly reused, but risky to modify
- Low NOC → limited reuse

Example of DIT and NOC

Code

```
class Animal {} //  
class Mammal extends Animal {}  
class Reptile extends Animal {}  
class Dog extends Mammal {}
```

Values

```
DIT = 0, NOC = 2  
// DIT = 1, NOC = 1  
// DIT = 1, NOC = 0  
// DIT = 2, NOC = 0
```

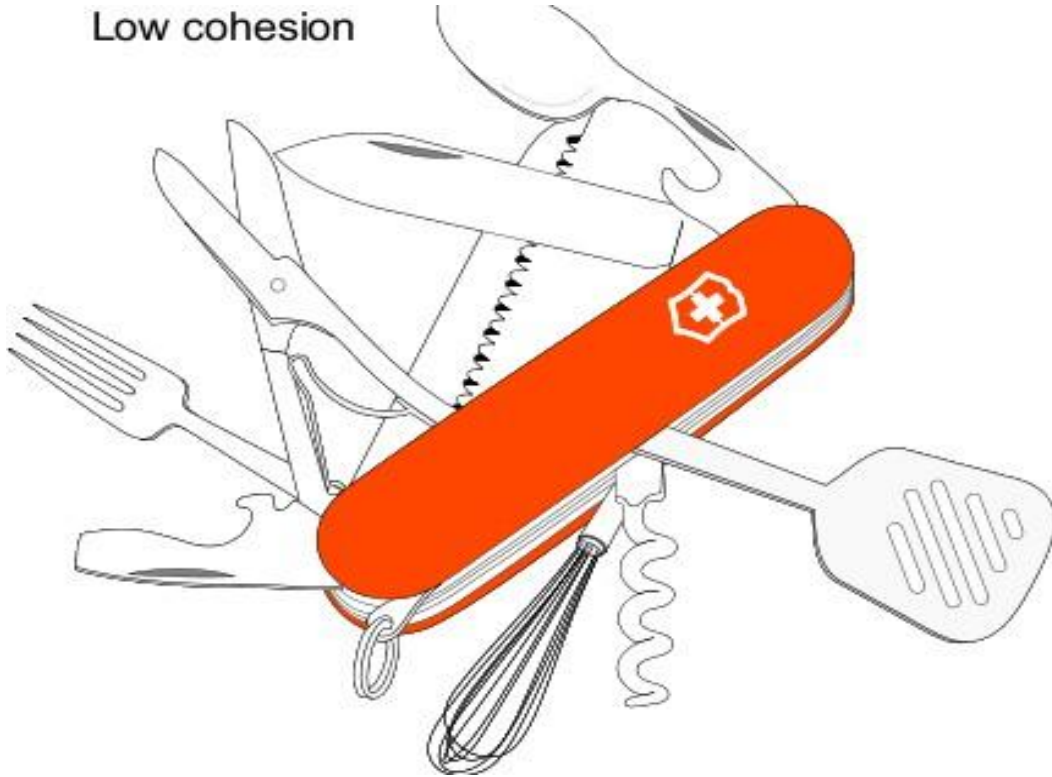
Design for High Cohesion

- Cohesion
 - Degree to which the elements of module belong together
 - Measured in ordinal sense i.e. low, high, ...
 - Cohesion is not “each method does its own task.”
- - Cohesion means: all methods together serve ONE well-defined responsibility and work on the SAME state.

Cohesion is decreased if:

- The functionalities embedded in a class, accessed through its methods, have little in common
- Methods carry out many varied activities, often using coarsely-grained or unrelated sets of data

Low cohesion

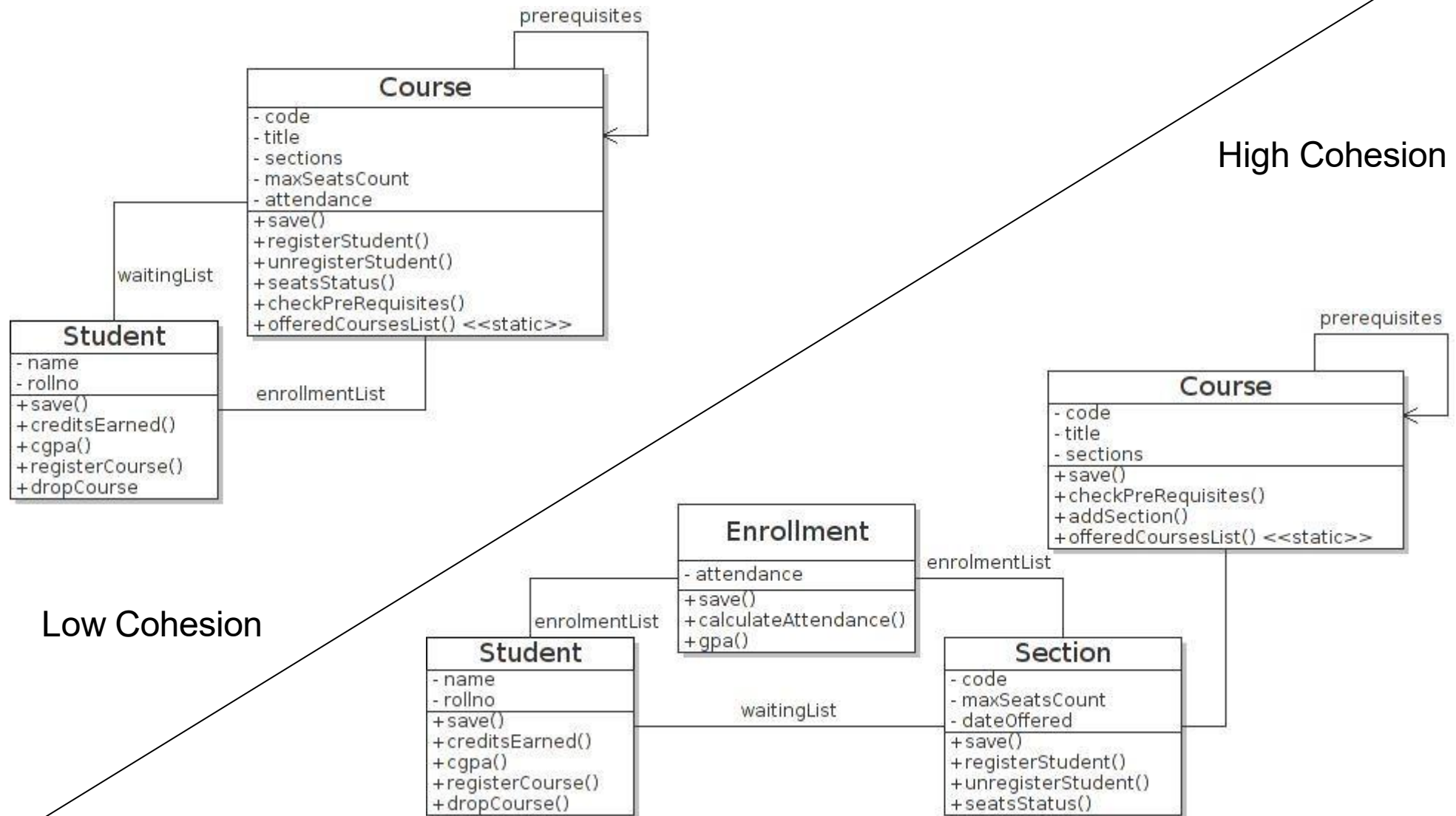


High cohesion



Taken from Software Engineering by Ivan Marsic

Example: Course Registration System



WMC (Weighted Methods per Class)

- WMC is the sum of the complexities of all methods in a class.If complexity is not specified, each method counts as 1.
- What WMC Measures
 - Class size
 - Class complexity
 - Effort required to maintain and test a class
- Low cohesive classes tend to be more complex and hence more difficult to maintain
- High WMC can indirectly be taken as an indicator of low cohesion
- Mathematical Definition:
 - No specific complexity measure is recommended but Cyclomatic Complexity is often used
 - Too many methods or too large methods results in high WMC, indicating low cohesion

WMC(code example) with cyclomatic complexity

```
class Calculator {  
    int add(int a, int b) { return a + b; }           // CC = 1  
  
    int max(int a, int b) {                           // CC = 2  
        if (a > b) return a;  
        else return b;  
    }  
}
```


Measuring (lack of) cohesion

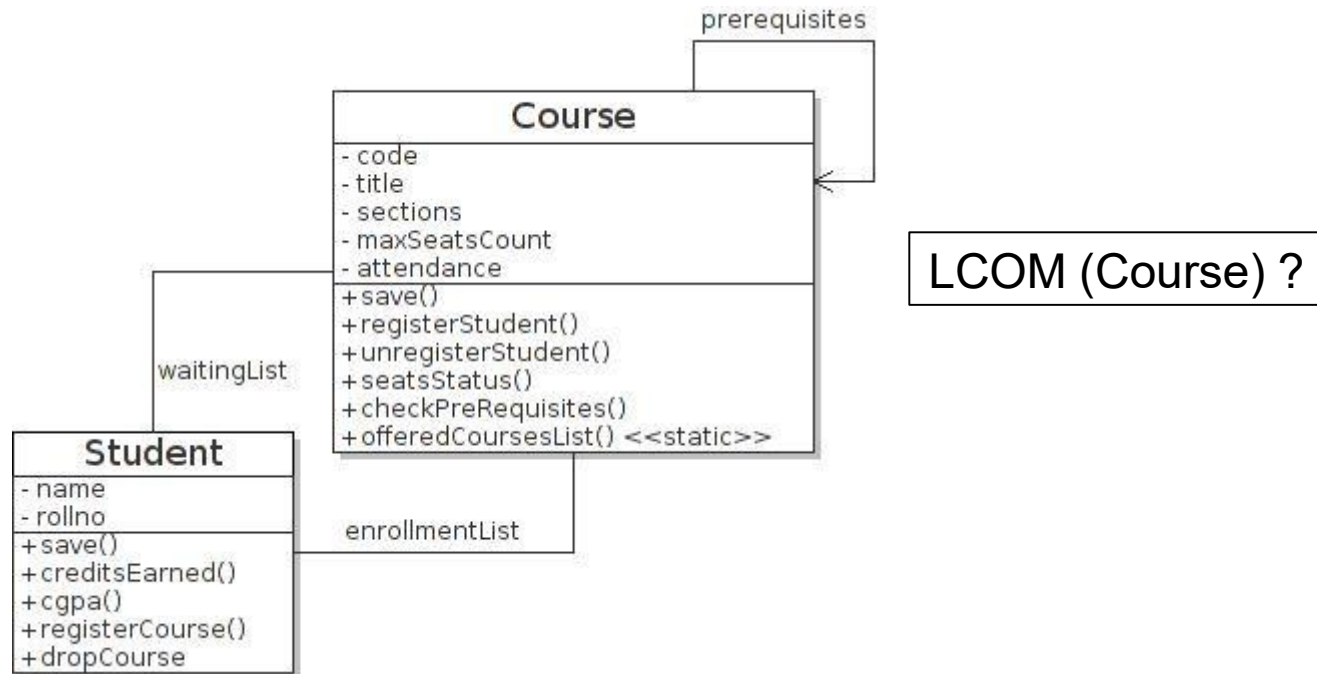
CK Metrics: LCOM

- Lack of cohesion in methods (LCOM)
 - Measures degree of similarity of methods (based upon the member variables they use)
 - **LCOM measures how closely related the methods of a class are based on shared instance variables.**
 - **P** = number of pairs of methods that **do not share** an instance variable.
 - **Q** = number of pairs of methods that **do share** an instance variable.
 - $|P|-|Q|$, if $|P|>|Q|$, 0 other wise
- High LCOM value, low cohesion, 0 means high cohesion

Measuring (lack of) cohesion

CK Metrics: LCOM

- Lack of cohesion in methods (LCOM)
 - Larger the number of similar methods, lower the value of LCOM, hence more cohesive the class
 - Lower the number of similar methods, higher the value of LCOM, hence less cohesive the class
 - Lack of cohesion implies class should be split into two or



LCOM example(Bad)

```
public class Employee {  
  
    private String name;  
    private double salary;  
    private String printerName;  
  
    public void updateName(String newName) {  
        name = newName;  
    }  
  
    public void giveBonus(double bonus) {  
        salary = salary + bonus;  
    }  
  
    public void printReport(String report) {  
        System.out.println(  
            "Printing " + report +  
            " using " + printerName  
        );  
    }  
}
```

LCOM example(Good)

```
public class BankAccount {  
  
    private double balance;  
  
    public void deposit(double amount) {  
        balance = balance + amount;  
    }  
  
    public void withdraw(double amount) {  
        if (amount <= balance) {  
            balance = balance - amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
  
    public void displayBalance() {  
        System.out.println("Balance: " + balance);  
    }  
}
```

LCOM variations

Henderson-Sellers LCOM definition:

LCOM(HS) measures **how well methods of a class use its attributes**.

Consider a set of m methods, $M_1, M_2, \dots, M_m$.

The methods access a data attributes, $A_1, A_2, \dots, A_a$.

Let $m(A_k)$ = number of methods that access data A_k .

Then,

$$LCOM^* = \frac{\left(\frac{1}{a} \sum_{j=1}^a m(A_j) \right) - m}{1 - m}$$

LCOM* varies from 0 to 1, where 0 indicating high cohesion and 1 indicating low cohesion

LCOM(HS) Example

```
class Order {  
    int orderId;  
    double amount;  
  
    void setOrderId(int id) {  
        orderId = id;  
    }  
  
    void setAmount(double amt) {  
        amount = amt;  
    }  
  
    double calculateDiscount() {  
        return amount * 0.1;  
    }  
  
    void printOrder() {  
        System.out.println(orderId + " " + amount);  
    }  
}
```

Attributes (a = 2)

- orderId
- amount

Methods (m = 4)

1. setOrderId()
2. setAmount()
3. calculateDiscount()
4. printOrder()

LCOM(HS) Example(Moderate Cohesion)

- **Attribute–Method Usage Table**

Attribute	Methods Using It	$\mu(A_i)$
orderId	setOrderId, printOrder	2
amount	setAmount, calculateDiscount, printOrder	3

- **Computations**

$$\sum \mu(A_i) = 2 + 3 = 5$$

$$\frac{1}{a} \sum \mu(A_i) = \frac{5}{2} = 2.5$$

$$\text{LCOM(HS)} = \frac{m - 2.5}{m - 1}$$

$$= \frac{4 - 2.5}{3}$$

$$= \frac{1.5}{3}$$

$$= 0.5$$

Interpretation

- Moderate cohesion
- Methods are partially related
- Acceptable design, but could be improved
- Class is not completely focused on one responsibility

LCOM(HS) Example

LCOM(HS) Value

0.0 – 0.3

0.3 – 0.6

0.6 – 1.0

Meaning

Highly cohesive

Medium cohesion

Low cohesion

Class Activity

Compute LCOM(HS) of the given `OnlineOrder` class

Conclusion?

Design for Low Coupling

- Coupling is the degree to which a module / class is dependent on another module / class to perform its operations
- Change in one leads to change in other
- Coupling cannot be avoided as such, but it should be reduced to minimum possible

Tight coupling

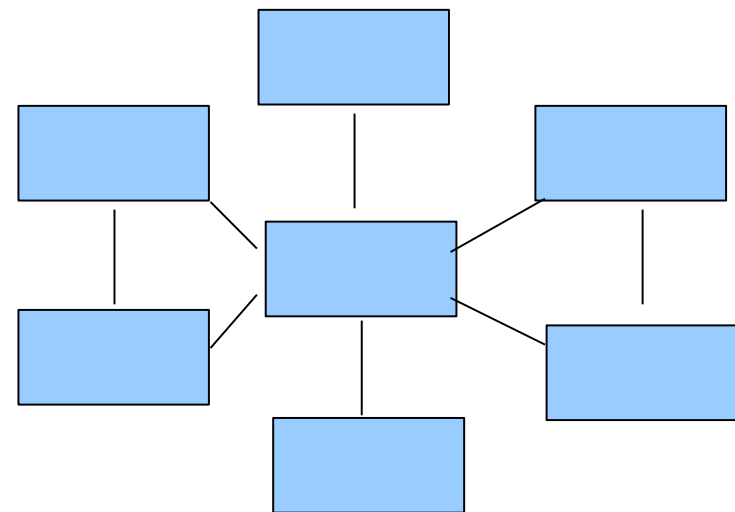
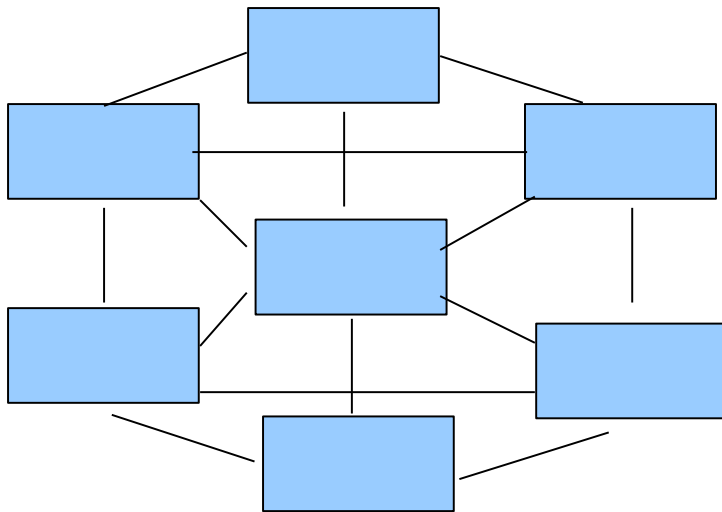


Loose coupling



Taken from Software Engineering by Ivan Marsic

High Coupling

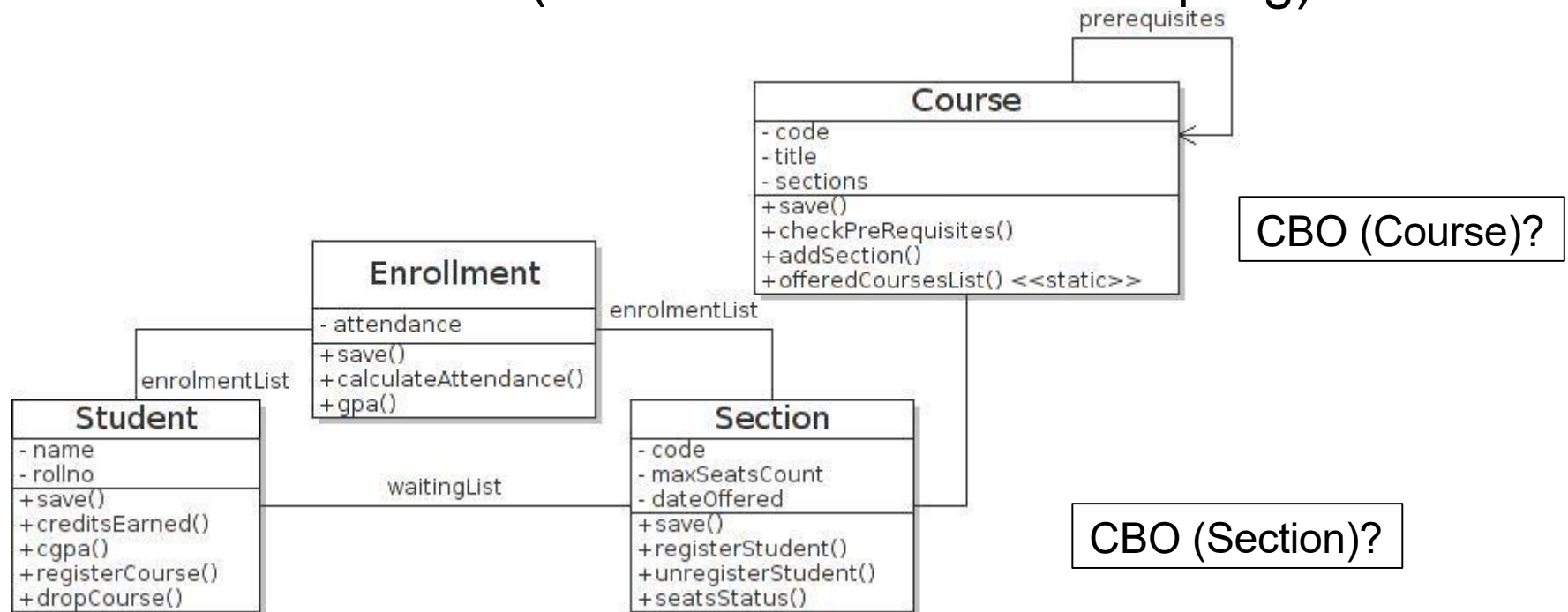


Low Coupling

Measuring coupling

CK Metrics: CBO

- Coupling between objects (CBO)
 - CBO for a class is a count of number of other classes to which it is coupled
 - Object is coupled to another if one of them acts on the other i.e. methods of one use methods or instance variables of other (measures Interaction coupling)



CBO

CBO = number of distinct classes a class is coupled to (excluding itself).

Coupling occurs via:

- Attribute types
- Method parameters
- Method calls to other classes

CBO Example

```
class Customer {  
    String name;  
  
    void setName(String n) {  
        name = n;  
    }  
  
    String getName() {  
        return name;  
    }  
}
```

```
class Order {  
    int orderId;  
    double amount;  
    Customer customer;  
  
    void setOrder(int id, double amt) {  
        orderId = id;  
        amount = amt;  
    }  
  
    double calculateTotal() {  
        return amount + (amount * 0.1);  
    }  
  
    void assignCustomer(Customer c) {  
        customer = c;  
        customer.getName(); // method call to another class  
    }  
}
```


CBO Example (cont)

CBO ?

```
class Invoice {  
    Order order;  
  
    void generateInvoice(Order o) {  
        order = o;  
        order.calculateTotal(); // method call to Order  
    }  
  
    void printInvoice() {  
        System.out.println("Invoice printed");  
    }  
}
```

Measuring coupling

CK Metrics: RFC

- Response for a class (RFC)
 - Measures the size of response set of a class
 - Response set of a class represents the methods that can be potentially executed in response to a message received by an object of that class
 - Comprise of all methods in class + all methods of other classes invoked in each method

$$RS = \{M\} \cup_{all \ i} \{R_i\}$$

where M = set of methods in class

R_i = set of methods called by method i in set M

LK Metrics

- Class Size (CS)
- Number of operations overridden (NOO)
- Number of operations added (NOA)
- Specialization Index (SI)
- Average operation size (OS_{avg})
- Operation complexity (OC)
- Average number of parameters (Np_{avg})

MOOD Metrics

- Method inheritance factor (MIF)
- Coupling factor (CF)
- Polymorphism factor (PF)